Security_Report

InvoicePortal - Challenge Security Report

9.1 Executive Summary

InvoicePortal is a Flask-based REST API lab that demonstrates a chained attack combining **Broken Authentication** (weak JWT signing secret) with **Broken Object Level Authorization (BOLA)**. A low-privileged user can crack the JWT secret offline, forge a token as another user, and access invoices that do not belong to them. The flag is hidden inside one of the victim's invoices and is only retrievable after successfully impersonating another user. This lab teaches the importance of strong secrets and per-object authorization checks in modern APIs.

9.2 Challenge Overview

Application: InvoicePortal is a SaaS-style platform used by Northwind Traders to manage company invoices and team members.

Scenario: A low-privileged employee (`attacker`) wants to access an invoice that belongs to another employee (`victim`) in the same company. The invoice contains an internal audit note that holds the flag.

Primary Vulnerability: Broken Object Level Authorization (BOLA) on `/api/invoices` and `/api/invoices/{id}`.

Secondary Vulnerability: Weak JWT singing secret (HS256) that can be cracked offline using small wordlist.

Difficulty: Medium

9.3 Architecture & Trust Boundaries

Components

- **Flask API:** (app.py) handles HTTP requests, JWT authentication, And invoices operations.

- **SQLite Database:** (/data/invoiceportal.db) stores users, companies, and invoices.
- **JWT middleware:** Verifies token signature and extracts the sub claim.

Data Flow

(Player)—HTTP → [Flask API] —SQL → SQLite

|

JWT middleware

|

/api/invoices

9.4 Attacker Start Point

- Initial Access: Valid user account (attacker/ attacker_pass123).
- **Role:** Low-privileged employee in the same company as victim.
- **Known Information:** Can call /api/team/members to discover victim's user ID.
- **Restrictions:** No admin access, no direct database access, cannot guess invoice UUIDs.

Attacker Surface

Endpoint	Method	Auth
/health	GET	None
/api/auth/login	POST	None
/api/users/me	GET	Bearer
/api/team/members	GET	Bearer
/api/invoices	GET	Bearer
/api/invoices/{id}	GET	Bearer

9.6 Vulnerability Description

1. Weak JWT Signing Secret (Broken Authentication)

- **Affected Component:** /api/auth/login, JWT middleware
- **Preconditions:** Attacker has any valid user account
- **Failed Control:** Cryptographic key management
- **Root Cause:** The secret (changeme2024) is hardcoded, short, and present in a common wordlist. HS256 is symmetric, so anyone who cracks the secret can sign arbitrary tokens.

2. Broken Object Level Authorization (BOLA)

- **Affected Component:** /api/invoices, /api/invoices/{id}
- **Preconditions:** Attacker can send requests with a valid JWT
- **Failed Control:** Per-object authorization check
- **Root Cause:** The API trusts the sub claim completely and never checks that the authenticated user owns the requested invoice.

9.7 Intended Attack Path

1. **Enumeration:** Discover /api/auth/login, /api/invoices, /api/team/members, /api/users/me.
2. **Login:** Authenticate as attacker and obtain a JWT.
3. **Analyze JWT:** Decode the token, observe alg: HS256 and sub: 2.
4. **Crack Secret:** Use hashcat -m 16500 with the provided wordlist to recover changeme2024.
5. **Reconnaissance:** Call /api/team/members to find victim's user_id (1).
6. **Forge Token:** Create a new JWT with sub: 1 signed with changeme2024.
7. **Exploit BOLA:** Call /api/invoices with the forged token. The API returns victim's invoices.
8. **Retrieve Flag:** Call /api/invoices/{id} for the invoice containing the flag in its notes field.

9.8 Impact Assessment

Confidentiality: (High) Full access to other user's invoices and sensitive financial data.

Integrity: (Medium) Attacker can potentially modify or delete invoices in a real-world app.

Availability: (Low) No denial of service demonstrated.

Privileges Gained: Horizontal privilege escalation (Attacker → victim).

9.9 Flag Retrieval

The attacker must present a valid JWT with sub matching the victim's (user_id) (1) and request the invoice that holds the flag.

How the Vulnerability leads to the flag?:

1. Weak JWT secret allows forgery.
2. Missing ownership check allows cross-user access.
3. The victim's invoice contains the flag in (notes) field.

9.10 Root Cause

The application fails at two security controls:

1. **Cryptographic Key Management:** A weak, hardcoded secret is used for JWT signing and is present in a public wordlist.
2. **Per-Object Authorization:** Object-level endpoints trust the sub claim from the JWT without verifying ownership at the database layer.

9.11 Remediation

1: Strong JWT Secret + Asymmetric Signing

- Generate a 256-bit secret using secrets.token_hex(32).
- Store it in an environment variable, never in source code.
- Refuse to start if the secret matches a known default.
- Prefer RS256 (asymmetric) so the public key cannot be used to forge tokens.

2: Enforce Ownership Checks

- Add AND owner_id = ? to every object-level query.
- Create a require_ownership decorator that validates the authenticated user against the object's owner.
- Return 403 Forbidden or 404 Not Found when the user is not the owner.
- Use UUIDs (already in place) plus ownership checks for defense in depth.

9.12 Verification & Reset

Verification

- Access victim's invoice with attacker's token
  - 200 OK with Data → vulnerable behavior
  - 403/404 → Secure Behavior
- Crack JWT token with wordlist
  - Secret recovered → Vulnerable behavior
  - Secret not in the list → Secure Behavior
- Forge token with guessed secret
  - Accepted (200 OK) → vulnerable behavior
  - Rejected (401) → Secure Behavior

Reset

1. Reset the lab with `docker compose down -v && docker compose up -d`.
2. Repeat the intended attack path — it should fail after remediation.
3. Run automated tests against the secure version

9.13 Unintended Attack Paths

During testing, no unintended attack paths were found:

- No SQL injection in any endpoint.

- No unauthenticated access to any protected resource.
- No IDOR in other endpoints (all are scoped by sub).
- No path traversal or file disclosure.
- UUIDs prevent invoice enumeration.

All endpoints were reviewed, and the intended chain is the only viable path to the flag.

9.14 Conclusion

InvoicePortal demonstrates a realistic chained attack that combines **Broken Authentication** with **Broken Object Level Authorization**. The lab is:

- **Reproducible:** Resettable with `docker compose down -v && docker compose up -d`.
- **Isolated:** No external dependencies.
- **Deterministic:** The flag is reachable through a single intended path.
- **Educational:** Teaches the importance of strong secrets and per-object authorization in APIs.

Limitations: The application is intentionally simple and does not model advanced API features (rate limiting, logging, monitoring). These would be added in a production-grade system.